

1)IMPORTING REQUIRED LIBRARIES

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import MultinomialNB, GaussianNB
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score
```

2.LOADING DATASET

```
df = pd.read_csv("emails.csv")
print(df.head())
print()
print(df.info())
```

		text	spam
0	Subject: naturally irresistible your corporate...		1
1	Subject: the stock trading gunslinger fanny i...		1
2	Subject: unbelievable new homes made easy im ...		1
3	Subject: 4 color printing special request add...		1
4	Subject: do not have money , get software cds ...		1

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5728 entries, 0 to 5727
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   text    5728 non-null     object  
 1   spam    5728 non-null     int64   
dtypes: int64(1), object(1)
memory usage: 89.6+ KB
None
```

3.DATA CLEANING AND PREPROCESSING

```
print(df.isnull().sum())

df.dropna(inplace=True)

print(df['spam'].value_counts())
```

```

text      0
spam      0
dtype: int64
spam
0      4360
1      1368
Name: count, dtype: int64

```

4.SPLIT DATASET INTO TRAIN AND TEST:

```

X = df['text']
y = df['spam']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

```

5.MODELS

MODEL 1:BAG OF WORDS + MULTINOMIAL NAIVE BAYES

Spam emails usually contain words like:

free, win, offer, prize, click, money, urgent

After Bag of Words, the model sees this:

Word	win	free	prize	project	meeting
Spam Email	1	1	1	0	0
Normal Email	0	0	0	0	1

MULTINOMIAL NAIVE BAYES:

Email contains high-weight spam words → classified as SPAM.

Focuses on meaningful spam words, not just word count.

```

# from sklearn.feature_extraction.text import TfidfVectorizer

# tfidf = TfidfVectorizer(stop_words='english')
# X_train_tfidf = tfidf.fit_transform(X_train)
# X_test_tfidf = tfidf.transform(X_test)

```

```

# mnb_tfidf = MultinomialNB()
# mnb_tfidf.fit(X_train_tfidf, y_train)

# y_pred_tfidf = mnb_tfidf.predict(X_test_tfidf)

# print("\nTF-IDF + Multinomial NB")
# print("Accuracy :", accuracy_score(y_test, y_pred_tfidf))
# print("Precision:", precision_score(y_test, y_pred_tfidf))
# print("Recall   :", recall_score(y_test, y_pred_tfidf))
# print("F1 Score :", f1_score(y_test, y_pred_tfidf))
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

bow = CountVectorizer(stop_words='english')

X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)

mnb_bow = MultinomialNB()
mnb_bow.fit(X_train_bow, y_train)

y_pred_bow = mnb_bow.predict(X_test_bow)

print("\nBag of Words + Multinomial NB")
print("Accuracy :", accuracy_score(y_test, y_pred_bow))
print("Precision:", precision_score(y_test, y_pred_bow))
print("Recall   :", recall_score(y_test, y_pred_bow))
print("F1 Score :", f1_score(y_test, y_pred_bow))

Bag of Words + Multinomial NB
Accuracy : 0.9886561954624782
Precision: 0.9792387543252595
Recall   : 0.9758620689655172
F1 Score : 0.9775474956822107

```

MODEL 2:TF-IDF + MULTINOMIAL NAIVE BAYES

TF-IDF : TERM FREQUENCY – INVERSE DOCUMENT FREQUENCY.

TF measures how often a word occurs in a single document.

IDF reduces the weight of words that occur in many documents, highlighting unique, meaningful words.

TF-IDF is a technique that gives higher weight to important words and lower weight to common words in text data.

*Word "free" → appears mainly in spam → high TF-IDF

*Word "the" → appears everywhere → low TF-IDF

Spam email:

"Win free prize now"

Normal email:

"Project meeting tomorrow"

TF-IDF:

Words like "win", "free", "prize" appear many times in spam emails but rarely in normal emails.

TF-IDF gives high weight to these words.

Common words like "now", "the", "is" get low weight.

MULTINOMIAL NAIVE BAYES:

Email contains high-weight spam words → classified as SPAM.

Focuses on meaningful spam words, not just word count.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(stop_words='english')
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

mnb_tfidf = MultinomialNB()
mnb_tfidf.fit(X_train_tfidf, y_train)

y_pred_tfidf = mnb_tfidf.predict(X_test_tfidf)

print("\nTF-IDF + Multinomial NB")
print("Accuracy :", accuracy_score(y_test, y_pred_tfidf))
print("Precision:", precision_score(y_test, y_pred_tfidf))
print("Recall   :", recall_score(y_test, y_pred_tfidf))
print("F1 Score :", f1_score(y_test, y_pred_tfidf))
```

```
TF-IDF + Multinomial NB
Accuracy : 0.8926701570680629
Precision: 1.0
```

```
Recall    : 0.5758620689655173
F1 Score  : 0.7308533916849015
```

MODEL 3: TF-IDF + GAUSSIAN NAIVE BAYES

"Free prize available"

What happens here:

TF-IDF converts words into decimal values like:

free → 0.87

prize → 0.82

Gaussian Naive Bayes assumes these values follow a normal (bell-shaped) distribution.

Why it fails:

Text data is sparse and uneven, not normally distributed.

TF-IDF values jump irregularly instead of forming smooth curves.

Result:

Gaussian NB may misclassify spam as not spam.

```
from sklearn.naive_bayes import GaussianNB

gnb = GaussianNB()
gnb.fit(X_train_tfidf.toarray(), y_train)

y_pred_gnb = gnb.predict(X_test_tfidf.toarray())

print("\nTF-IDF + Gaussian NB")
print("Accuracy :", accuracy_score(y_test, y_pred_gnb))
print("Precision:", precision_score(y_test, y_pred_gnb))
print("Recall    :", recall_score(y_test, y_pred_gnb))
print("F1 Score  :", f1_score(y_test, y_pred_gnb))
```

```
TF-IDF + Gaussian NB
Accuracy : 0.9537521815008726
Precision: 0.9575289575289575
Recall    : 0.8551724137931035
F1 Score  : 0.9034608378870674
```

Why Accuracy Can Be Misleading in Spam Detection:

Suppose your dataset is 90% non-spam (ham) and 10% spam.

A dumb model that predicts everything as non-spam gets 90% accuracy – sounds great, but it catches 0% spam.

This shows that accuracy alone doesn't reflect the model's ability to detect spam.

In Multinomial NB model, Precision = 1.0, meaning every email predicted as spam is actually spam.

1. BEST MODEL:

Spam email detection was implemented successfully using Naive Bayes classifiers.

Bag of Words + Multinomial Naive Bayes works well because it uses word counts to identify spam patterns.

TF-IDF + Multinomial Naive Bayes gives the best performance because important spam-related words get higher weight and common words get less importance.

Gaussian Naive Bayes performs poorly for text data because it is designed for continuous numerical data, not word-based features.

Overall, TF-IDF + Multinomial Naive Bayes is the most suitable and accurate model for spam email classification.

1. USECASES:

Use Case: Spam Detection in Emails

1. Detects unwanted or suspicious emails automatically.
2. Moves spam emails to a separate folder so the inbox stays clean.
3. Protects users from phishing, scams, and malware.
4. Learns from user feedback to improve accuracy over time.
5. Saves time and improves email experience for users.

Where it is used:

Email services like Gmail, Outlook, Yahoo Mail

Messaging platforms with automated spam filters

1. CONCLUSION:

This project successfully created a system to detect spam emails. It helps in automatically filtering unwanted messages, keeping inboxes clean and safe. The system improves email experience by saving time and protecting users from scams.